

PROJECT REPORT

Hospital Virtual Appointment Booking Queue System

Submitted To: Department of Computer Science and Information Technology Pokhara University

Submitted By: Student Name: Aayushma Joshi Roll No.: 24080602 College: Rajdhani Model College

Submitted To: Project Supervisor: Academic Year: 2026/20

Certificate

This is to certify that the project entitled "**Hospital Virtual Appointment Booking Queue System**" submitted by **Aayushma Joshi** in partial fulfillment of the requirements for the degree of **Bachelor of Computer System and Information Technology (BCSIT)**, **Pokhara University**, is an original work carried out under my supervision.

Supervisor's Signature: _____

Date: _____

Acknowledgement

I would like to express my sincere gratitude to my project supervisor for the continuous guidance, valuable suggestions, and unwavering encouragement provided throughout the development of the Hospital Virtual Appointment Booking Queue System. The expertise and constructive feedback I received played a significant role in the successful completion

of this project.

I am also deeply thankful to the faculty members of the Department of Computer Science and Information Technology at Rajdhani Model College for imparting the foundational knowledge and technical skills necessary to undertake a software development project of this scale.

Furthermore, I extend my appreciation to my friends and family for their endless moral support and motivation during the challenging phases of system design and implementation. This project would not have been possible without their collective support.

Abstract

The Hospital Virtual Appointment Booking Queue System is a comprehensive, web-based application engineered to digitalize and streamline the appointment scheduling and queue management processes within healthcare facilities. Traditionally, hospitals rely on manual or disjointed digital systems to manage patient inflow, which frequently results in overcrowded waiting rooms, unpredictable waiting times, and administrative bottlenecks. This project addresses these critical inefficiencies by introducing an automated platform that manages the entire lifecycle of a patient's outpatient visit.

Developed utilizing a robust technology stack—including Spring Boot for backend logic, Spring Security for role-based access control, Spring Data JPA with MySQL for persistent data storage, and Thymeleaf integrated with Tailwind CSS for a responsive frontend—the system delivers a seamless experience across three distinct user roles: Patient, Doctor, and Administrator. Patients can register, browse available medical professionals, book appointments, and monitor their real-time queue status, complete with algorithmically estimated waiting times. Doctors are equipped with tools to manage their daily schedules, control patient flow by updating queue statuses (e.g., Waiting, In Progress, Completed), and review comprehensive patient histories. Administrators oversee the holistic operation of the platform, managing users, generating analytical reports, and ensuring system integrity.

By implementing algorithmic queue generation and priority scoring (distinguishing between normal and emergency appointments), the system significantly optimizes patient throughput. The integration of modern UI components ensures accessibility across mobile and desktop devices. Ultimately, the Hospital Virtual Appointment Booking Queue System

minimizes physical congestion in healthcare facilities, enhances the accuracy of medical administrative records, and substantially improves the overall healthcare experience for both providers and patients.

Table of Contents

1. [Chapter 1: Introduction](#)
 2. [Chapter 2: Problem Statement](#)
 3. [Chapter 3: Objectives](#)
 4. [Chapter 4: Scope of the Project](#)
 5. [Chapter 5: Literature Review](#)
 6. [Chapter 6: System Analysis](#)
 7. [Chapter 7: Feasibility Study](#)
 8. [Chapter 8: Requirement Analysis](#)
 9. [Chapter 9: System Design](#)
 10. [Chapter 10: Database Design](#)
 11. [Chapter 11: Implementation](#)
 12. [Chapter 12: Testing](#)
 13. [Chapter 13: Results](#)
 14. [Chapter 14: Conclusion](#)
 15. [Chapter 15: Future Enhancements](#)
 16. [Chapter 16: References](#)
-

Chapter 1

Introduction

The rapid advancement of information technology has fundamentally transformed various sectors, with healthcare being one of the primary beneficiaries. Despite these technological strides, many healthcare institutions, particularly in developing regions, continue to struggle with outpatient department (OPD) management. The traditional

process of visiting a hospital, waiting in long physical queues to book an appointment, and subsequently waiting outside a doctor's cabin for an indeterminate amount of time is highly inefficient. It induces immense stress on patients, who are often already in a state of physical distress, and imposes a heavy administrative burden on hospital staff.

The Hospital Virtual Appointment Booking Queue System is a web-based software solution designed to eliminate the friction associated with medical appointments. By shifting the queueing paradigm from a physical space to a virtual platform, the system allows patients to secure their position in a doctor's schedule remotely. It operates on the principle of providing transparent, real-time data to all stakeholders involved in the medical consultation process.

For patients, the platform serves as a centralized portal to view doctor availability, book appointments, and track their dynamic position in the queue. The system actively computes estimated waiting times based on the current progression of the queue, allowing patients to arrive at the hospital precisely when they are needed. For medical professionals, the software provides a digitized daily itinerary, complete with patient medical summaries and one-click queue management tools, allowing them to focus entirely on patient care rather than logistical coordination. For hospital administration, the system offers a macroscopic view of hospital operations, providing analytical insights into patient volume, departmental efficiency, and staff allocation.

Built upon the enterprise-grade Spring Boot framework, the application emphasizes reliability, data security, and scalability. It incorporates role-based access control (RBAC) to ensure strict adherence to data privacy standards, ensuring that sensitive medical information is only accessible to authorized personnel. Through the digitization of the appointment workflow, the Hospital Virtual Appointment Booking Queue System aims to foster a more organized, efficient, and patient-centric healthcare environment.

Chapter 2

Problem Statement

The administration of patient flow in modern healthcare facilities is fraught with logistical challenges. The reliance on manual, paper-based ledgers or rudimentary, disconnected software systems yields several critical problems that degrade both the quality of

healthcare delivery and the patient experience. The core issues prevalent in the current operational models include:

1. **Unpredictable Waiting Times and Overcrowding:** Patients are frequently required to arrive at the hospital hours before their actual consultation. Without a transparent tracking mechanism, waiting rooms become severely overcrowded. This not only causes immense discomfort to patients but also increases the risk of cross-infection, a vulnerability that was acutely highlighted during recent global health crises.
2. **Inefficient Handling of Emergencies:** In manual queueing systems, accommodating walk-in emergency patients disrupts the entire daily schedule. Staff must manually adjust the queue, often leading to confusion, disputes among waiting patients, and delayed care for those requiring immediate attention.
3. **High Rate of Administrative Errors:** The manual transcription of patient details, appointment dates, and time slots frequently leads to human error. Double-bookings, lost appointment records, and miscommunication regarding doctor availability are common occurrences that directly impact hospital reputation and operational efficiency.
4. **Lack of Real-Time Communication:** Patients are entirely disconnected from the hospital's internal schedule once an appointment is booked. If a doctor is delayed due to an emergency surgery or prior consultations running over time, patients are not notified, leading to frustration and wasted time.
5. **Absence of Centralized Analytics:** Hospital administrators lack real-time visibility into daily operations. Extracting meaningful data regarding peak consultation hours, doctor workload distribution, and patient demographics requires manually compiling data from disparate sources, hindering effective resource allocation and strategic planning.

These collective challenges necessitate the transition from a physical, manual management paradigm to a virtual, automated system that prioritizes transparency, efficiency, and data integrity.

Chapter 3

Objectives

The primary aim of this software project is to develop a robust, scalable, and user-friendly platform that resolves the inefficiencies inherent in traditional hospital appointment

management. The objectives are categorized into a general overarching goal and specific functional targets.

General Objective

To design, develop, and deploy a computerized Hospital Virtual Appointment Booking Queue System that comprehensively manages the outpatient workflow, seamlessly connecting patients, doctors, and hospital administrators through a centralized virtual platform.

Specific Objectives

- **Digitalize Patient Onboarding:** To provide a secure registration and authentication module for patients to create and manage their medical profiles.
 - **Streamline Appointment Scheduling:** To implement an intuitive interface that allows patients to filter doctors by specialization, view available time slots, and independently book appointments.
 - **Automate Queue Management:** To develop an algorithm that automatically generates queue numbers based on appointment times and dynamically calculates estimated waiting times based on real-time queue progression.
 - **Implement Priority Routing:** To integrate a priority scoring system that automatically elevates emergency appointments to the top of the queue without manual intervention.
 - **Empower Medical Staff:** To equip doctors with a dedicated dashboard to view their daily schedule, access patient medical histories, and manually update the status of the ongoing consultation (e.g., In Progress, Completed).
 - **Enhance Administrative Oversight:** To build an administrative control panel capable of managing all user accounts (patients and doctors), overseeing global queues, and generating statistical reports for hospital management.
 - **Ensure Data Security:** To implement robust authentication and authorization mechanisms (using Spring Security) to protect sensitive healthcare data and ensure strict role-based access.
-

Chapter 4

Scope of the Project

The scope of the Hospital Virtual Appointment Booking Queue System encompasses the end-to-end management of outpatient appointments and the real-time handling of the resulting patient queues. The project is specifically bounded by the following modules and functionalities:

- **User Authentication and Authorization:** Secure login systems utilizing encrypted passwords, session management, and strict separation of privileges among three distinct roles: Admin, Doctor, and Patient.
- **Patient Module:** Functionalities allowing users to update demographic data, browse the hospital's directory of doctors, submit appointment requests (specifying normal or emergency types), view their live queue position, and cancel or reschedule appointments.
- **Doctor Module:** A tailored interface for physicians to toggle their availability status, view a chronological list of the day's appointments, access brief patient medical contexts, and trigger state changes in the queue (Starting, Completing, or Canceling a consultation).
- **Queue Generation Engine:** The backend logic responsible for assigning sequential queue numbers per doctor per day, calculating priority scores to sort the queue, and computing estimated wait times (assuming a standard consultation duration metric).
- **Administrator Module:** A comprehensive backend suite allowing hospital staff to onboard new doctors, deactivate user accounts, forcefully reschedule or cancel appointments in case of hospital emergencies, and monitor live queues across all departments.
- **Dashboard and Analytics:** Visual representations of data, including daily appointment counts, total registered users, and active queue statuses, providing quick insights to both doctors and administrators.

Out of Scope: The current iteration of the system does not include an integrated payment gateway for consultation fees, a pharmacy inventory management system, or a full Electronic Health Record (EHR) module for charting extensive medical diagnoses. It is strictly focused on the logistical routing of patients to doctors.

Chapter 5

Literature Review

The digitization of healthcare administration has been a focal point of software engineering research for the past two decades. Early iterations of Hospital Information Systems (HIS) were primarily focused on internal billing and inventory management, leaving patient-facing scheduling as a manual task. As internet accessibility proliferated, web-based appointment systems began to emerge.

A study on web-based appointment scheduling systems indicates that digital booking significantly reduces patient "no-show" rates and optimizes clinic capacity utilization. However, many existing systems operate purely as calendars; they allow a user to book a slot (e.g., 10:00 AM) but fail to account for the dynamic nature of medical consultations, which frequently run over their allotted time. When a 10:00 AM appointment stretches to 10:30 AM, subsequent patients are left waiting blindly, effectively reverting the digital system back to a physical, frustrating queue.

Modern research emphasizes the necessity of combining appointment scheduling with dynamic queue management. Systems that incorporate real-time queue tracking—often referred to as Virtual Queuing Systems (VQS)—have proven to drastically improve patient satisfaction scores. By allowing patients to monitor the queue remotely via web applications, they can utilize their waiting time productively outside the hospital environment, reducing physical crowding and the associated stress.

Commercially available systems often charge exorbitant licensing fees, rendering them inaccessible to smaller clinics or public hospitals in developing nations. Furthermore, many generic booking applications lack the specific nuances required for healthcare, such as prioritizing emergency triage over standard check-ups.

This project builds upon the established literature by proposing a system that specifically marries calendar-based booking with dynamic, priority-aware virtual queuing. By utilizing an open-source, enterprise-grade technology stack (Spring Boot and MySQL), the proposed system offers a scalable, cost-effective, and highly specialized solution tailored for healthcare environments, addressing both the scheduling and the real-time flow control of patients.

Chapter 6

System Analysis

System analysis involves a detailed evaluation of the current operational methods to identify bottlenecks and the subsequent conceptualization of a software solution designed to rectify these issues.

Existing System

In the existing manual or semi-automated system, patients typically secure appointments via telephone calls or by physically visiting the hospital reception. The receptionist logs the appointment in a physical ledger or a basic spreadsheet. On the day of the appointment, patients arrive, collect a physical token, and sit in a waiting area. The doctor's assistant manually calls out token numbers. If an emergency patient arrives, the assistant must verbally negotiate with waiting patients to prioritize the emergency case.

Problems in Existing System

- **Lack of Transparency:** Patients have no visibility into how many people are ahead of them or how long they will be required to wait.
- **Physical Congestion:** Because patients must remain within earshot of the doctor's cabin to hear their name called, waiting areas become severely congested.
- **Administrative Overhead:** Reception staff spend a disproportionate amount of time answering phone calls merely to confirm appointment timings or inform patients of delays.
- **Static Priority Handling:** Manually prioritizing emergency cases is prone to human bias, error, and often leads to disputes among waiting patients.
- **Data Fragmentation:** Patient records and appointment histories are siloed in physical files, making it difficult for doctors to retrieve past consultation data quickly.

Proposed System

The proposed Hospital Virtual Appointment Booking Queue System is a centralized web application that automates the entire workflow. Patients create accounts and book their own appointments online, selecting their preferred doctor and time slot. The system's backend automatically assigns a queue number and calculates an estimated wait time. On the day of the appointment, the patient can check their phone or computer to see the live queue progressing. The doctor interacts with a tablet or computer in their office, clicking a button to call the next patient. The system automatically handles the complex logic of placing emergency bookings at the top of the queue.

Advantages

- **Remote Waiting:** Patients can wait at home or in their vehicles until their queue number approaches, minimizing hospital crowding.
 - **Automated Triage:** The programmatic priority scoring ensures emergency patients are seamlessly and fairly prioritized without manual intervention.
 - **Operational Efficiency:** Reception staff are freed from constant phone calls and manual ledger management, allowing them to focus on critical administrative tasks.
 - **Data Accessibility:** Doctors have instant access to a digital list of today's patients, complete with their appointment context, improving the speed and quality of consultations.
 - **Scalability:** The centralized database approach allows the hospital to easily add new departments, doctors, and users without changing the underlying architecture.
-

Chapter 7

Feasibility Study

Before commencing the development phase, a comprehensive feasibility study was conducted to evaluate the practicality, viability, and potential success of the proposed system across four primary dimensions.

Technical Feasibility

The project is highly technically feasible. The selected technology stack (Java, Spring Boot, MySQL, Thymeleaf) is well-established, thoroughly documented, and actively maintained by large open-source communities. Spring Boot is specifically designed to handle enterprise-level concurrency, making it perfectly suited for a multi-user environment where patients, doctors, and admins are interacting with the database simultaneously. The deployment requirements are standard, requiring only a basic Java Runtime Environment and a relational database server, which can be hosted on standard local servers or cloud infrastructure.

Economic Feasibility

The proposed system is economically feasible. The development utilizes entirely free and open-source software (FOSS). Spring Boot, MySQL, and Tailwind CSS do not require commercial licensing for development. The primary costs associated with the project are

limited to development time and, eventually, server hosting costs. By significantly reducing the administrative workload on hospital reception staff and optimizing doctor scheduling, the system promises a high return on investment (ROI) through improved operational efficiency and patient throughput.

Operational Feasibility

Operationally, the system is designed to be highly intuitive. The frontend utilizes standard UI/UX paradigms (such as recognizable dashboards, simple forms, and clear action buttons), minimizing the learning curve for hospital staff and patients. For doctors, the system replaces manual paperwork with simple point-and-click actions to manage the queue. Furthermore, the inclusion of responsive design ensures that patients can access the platform via their mobile devices without needing to download a dedicated mobile application, thereby lowering the barrier to entry and ensuring high user adoption rates.

Legal Feasibility

The system is legally feasible. While healthcare applications are subject to strict data privacy regulations, the architecture addresses these concerns through robust security implementations. Passwords are cryptographically hashed using BCrypt, and Spring Security enforces strict session management and route protection, ensuring that unauthorized users cannot access patient data. As long as the hospital deploys the system on secure, compliant servers (with SSL encryption), the application meets the foundational requirements for data protection.

Chapter 8

Requirement Analysis

Requirement analysis defines the exact technical and functional specifications that the software must fulfill to be considered complete and successful.

Functional Requirements

Functional requirements define the specific behaviors, operations, and interactions that the system must support.

1. Authentication and Authorization Module - The system must allow users to register as patients. - The system must authenticate users using email and encrypted passwords. - The system must route authenticated users to role-specific dashboards (Admin, Doctor, Patient) based on their assigned authority.

2. Patient Capabilities - Patients must be able to view and update their personal profile (phone number, date of birth, blood group). - Patients must be able to view a list of all active doctors and their specializations. - Patients must be able to book an appointment by selecting a doctor, date, time slot, and specifying the appointment type (Normal/Emergency). - Patients must be able to view their upcoming and past appointments. - Patients must be able to cancel or reschedule their own appointments. - Patients must be able to view their real-time queue position and estimated waiting time for appointments scheduled on the current day.

3. Doctor Capabilities - Doctors must be able to toggle their availability status (Available/Unavailable). - Doctors must be able to view a real-time list (queue) of patients booked for the current day. - Doctors must be able to change the status of a queue item (Start, Complete, Cancel, Reschedule). - Doctors must be able to click on a patient's name to view their demographic details and appointment context.

4. Administrator Capabilities - Admins must have access to a statistical dashboard summarizing total users, appointments, and active queues. - Admins must be able to create, view, update, and delete Doctor profiles. - Admins must be able to view and delete Patient profiles. - Admins must be able to view, cancel, or reschedule any appointment across the entire system. - Admins must be able to view all active queues across all departments in real-time.

Non-Functional Requirements

Non-functional requirements dictate system attributes such as performance, usability, and security.

- **Security:** The system must prevent unauthorized access to URLs; e.g., a patient attempting to access /admin/dashboard must be rejected (HTTP 403). Passwords must not be stored in plain text.
- **Responsiveness:** The user interface must automatically adapt to different screen sizes, ensuring full functionality on mobile phones, tablets, and desktop monitors.
- **Performance:** The system should process appointment bookings and queue updates with minimal latency (under 2 seconds) to ensure the live queue reflects reality accurately.

- **Usability:** The interface must support clear visual feedback, utilizing toast notifications (via Toastify) to alert users of successful actions or errors without requiring page reloads.

Software Requirements

- **Operating System:** Windows, Linux, or macOS for development and deployment.
- **Java Development Kit (JDK):** Version 17 or higher (Java 21 recommended).
- **Framework:** Spring Boot 3.x.
- **Database:** MySQL Server 8.0+.
- **Build Tool:** Apache Maven.
- **Web Browser:** Modern browsers (Google Chrome, Mozilla Firefox, Safari, Edge).

Hardware Requirements

- **Processor:** Dual-core processor (Intel Core i3 or equivalent) minimum.
 - **RAM:** Minimum 4 GB (8 GB recommended for development).
 - **Storage:** Minimum 500 MB of free disk space for application files (database storage requirements scale with usage).
-

Chapter 9

System Design

System design translates the requirements into a blueprint for constructing the software, detailing the architecture, components, and data flow.

System Architecture

The application is built using the robust **Model-View-Controller (MVC)** architectural pattern provided by Spring Boot.

- **Model (Data Layer):** Represents the data structures (Entities) such as User, Patient, Doctor, Appointment, and Queue. These entities are mapped directly to MySQL database tables using Spring Data JPA (Hibernate), facilitating Object-Relational Mapping (ORM).

- **View (Presentation Layer):** The user interface is rendered server-side using the Thymeleaf templating engine. HTML templates are dynamically populated with data passed from the Controllers. Tailwind CSS is used for styling, and vanilla JavaScript (with libraries like Toastify) handles client-side interactivity.
- **Controller (Business Logic Layer):** Java classes annotated with `@Controller` intercept incoming HTTP requests, process business logic (often delegating to `@Service` components), interact with the database via Repositories, and return the appropriate View along with the necessary data model.

Workflow

1. A client (browser) sends an HTTP request (e.g., booking an appointment).
2. The request is intercepted by Spring Security to verify authentication and authorization.
3. The request is routed to the appropriate Controller method.
4. The Controller processes the input, interacts with the Repository layer to perform CRUD operations on the MySQL database.
5. The Database returns the requested data (or confirms the transaction).
6. The Controller populates the Thymeleaf Model with the data and specifies the template name.
7. Thymeleaf processes the template, generating standard HTML.
8. The HTML is returned as an HTTP response to the client's browser.

User Roles

The system strictly segregates functionality based on three roles: - **ROLE_ADMIN:** Complete oversight of the system. Manages staff and global data. - **ROLE_DOCTOR:** Restricted to viewing their own appointments, managing their specific queue, and viewing patient details assigned to them. - **ROLE_PATIENT:** Restricted to managing their own profile, booking appointments for themselves, and viewing their own queue status.

Use Case Description

Use Case: Book Appointment - **Actor:** Patient - **Precondition:** Patient is logged into the system. - **Main Flow:** Patient navigates to the booking page. The system displays a list of available doctors. The patient selects a doctor, chooses a date from the calendar, selects an available time slot, specifies the appointment type (Normal/Emergency), and submits the form. - **System Action:** The system validates the inputs, creates an Appointment record in the database. If the appointment date is today, the system calculates the priority score, assigns a queue number, and generates a Queue record. - **Postcondition:** The appointment is saved, and the patient is redirected to their dashboard with a success

notification.

Activity Flow (Queue Processing)

1. **Patient Arrival (Virtual):** When an appointment is booked for the current day, it enters the queue with status `WAITING`.
2. **Sorting Logic:** The system continuously sorts the queue. It first orders by `priorityScore` in descending order (Emergencies = 1, Normal = 0), and then by `queueNumber` in ascending order. This ensures emergencies always float to the top.
3. **Doctor Action - Start:** The doctor views the top patient and clicks "Start". The queue status changes to `IN_PROGRESS`.
4. **Doctor Action - Complete:** Upon finishing the consultation, the doctor clicks "Complete". The queue and appointment statuses are marked `COMPLETED`, and the record is removed from the active view. The next patient in line recalculates their estimated wait time.

Sequence Description (Authentication)

1. User enters credentials on the login page and submits.
 2. The request hits the Spring Security filter chain.
 3. The `CustomUserDetailsService` queries the `UserRepository` to find the user by email.
 4. The system compares the hashed password in the database with the provided password using `BCryptPasswordEncoder`.
 5. Upon success, a secure session is established, and the `AuthenticationSuccessHandler` redirects the user to their role-specific dashboard based on their granted authorities.
-

Chapter 10

Database Design

The database is designed using a relational model, ensuring data integrity, minimizing redundancy, and supporting the complex relational queries required for queue management.

ER Diagram Description

The Entity-Relationship architecture centers around the Appointment entity, which acts as a bridge between Patient and Doctor. - A **User** has a one-to-one relationship with either a **Patient** or a **Doctor**. - A **Patient** can have multiple **Appointments** (One-to-Many). - A **Doctor** can have multiple **Appointments** (One-to-Many). - An **Appointment** has a one-to-one relationship with a **Queue** entity (a queue record only exists for active, current-day appointments).

Database Tables and Relationships

1. users Table

2. Stores authentication credentials and role definitions.

Columns: id (PK), email (Unique), password, role (ADMIN, DOCTOR, PATIENT).

patients Table

5. Stores demographic information for patients.

Columns: id (PK), user_id (FK to users), first_name, last_name, phone, dob, blood_group.

doctors Table

8. Stores professional details of medical staff.

Columns: id (PK), user_id (FK to users), first_name, last_name, specialization, phone, is_available (Boolean).

appointments Table

11. Represents a scheduled consultation.

Columns: id (PK), patient_id (FK to patients), doctor_id (FK to doctors), appointment_date, time_slot, status (CONFIRMED, COMPLETED, CANCELLED), type (NORMAL, EMERGENCY), reason.

queues Table

14. Represents the volatile, real-time waiting line for a specific day.

15. *Columns:* id (PK), appointment_id (FK to appointments), queue_number (Integer), priority_level (String), priority_score (Integer), status (WAITING, IN_PROGRESS, COMPLETED, CANCELLED), estimated_waiting_time (Integer in minutes).

Data Dictionary

Table	Field	Data Type	Constraint	Description
users	email	VARCHAR(255)	UNIQUE, NOT NULL	Login identifier
users	role	VARCHAR(50)	NOT NULL	Defines RBAC authority
appointments	appointment_date	DATE	NOT NULL	Scheduled date of visit
appointments	time_slot	TIME	NOT NULL	Scheduled time block
queues	priority_score	INT	NOT NULL	1 for Emergency, 0 for Normal
queues	queue_number	INT	NOT NULL	Sequential integer for daily ordering

Chapter 11

Implementation

The implementation phase involved writing the source code and configuring the infrastructure to bring the system design to life. The project leverages a modern Java ecosystem designed for rapid, secure, and maintainable web development.

Technologies Used and Justification

1. Spring Boot (Java 21) Spring Boot was selected as the core backend framework. It drastically reduces boilerplate configuration required in traditional Java enterprise applications by providing auto-configuration and embedded servers (Tomcat). It was chosen for its unparalleled ecosystem, robust dependency injection, and ability to easily scale. Java 21 was utilized to leverage modern language features and performance enhancements.

2. Spring MVC The Model-View-Controller paradigm provided by Spring Web facilitates a clean separation of concerns. It handles the routing of HTTP requests, extraction of URL parameters and form data, and the orchestration of backend services before returning views. This modularity makes the codebase highly maintainable.

3. Spring Data JPA (Hibernate) Writing raw SQL queries for complex relational data is prone to errors and security vulnerabilities (like SQL injection). Spring Data JPA abstracts the database layer, allowing developers to interact with the database using Java objects (Entities). Methods like `findByAppointmentDoctorOrderByPriorityScoreDescQueueNumberAsc` are automatically translated into optimized SQL queries by the framework, significantly speeding up development time.

4. Spring Security Security is paramount in healthcare applications. Spring Security was implemented to handle user authentication, password hashing (BCrypt), session management, and route protection. It effortlessly ensures that methods and endpoints are strictly restricted based on the user's role, preventing privilege escalation attacks.

5. MySQL MySQL was selected as the relational database management system for production. It is ACID-compliant, highly reliable, and excels at managing structured relational data (like the strict connections between Patients, Doctors, and Appointments).

6. Thymeleaf Thymeleaf is a modern server-side Java template engine. Unlike older technologies like JSP, Thymeleaf templates are natural HTML files that can be viewed in a browser even without the backend running. It seamlessly integrates with Spring, allowing dynamic injection of model data into the HTML structure.

7. HTML, CSS, JavaScript & Tailwind CSS The frontend was constructed using semantic HTML5 and vanilla JavaScript for DOM manipulation (such as the client-side table search functionality). **Tailwind CSS**, a utility-first CSS framework, was utilized for styling. Tailwind allowed for rapid UI prototyping directly in the HTML, ensuring a consistent, modern, and highly responsive design without the bloat of traditional CSS files.

8. Toastify JS To enhance the user experience, Toastify was integrated to provide elegant, non-intrusive toast notifications for success and error messages, replacing jarring browser alert boxes.

Key Implementation Details

A critical implementation challenge was the queue sorting logic. By adding a `priorityScore` integer to the Queue entity, the complex logic of moving emergency patients to the front was reduced to a simple JPA sorting algorithm. When a new queue item is created, the repository simply retrieves the highest existing `queueNumber` for that doctor, increments it by 1, and assigns it. The display query then orders by `priorityScore DESC` followed by `queueNumber ASC`, naturally resulting in the correct triage order.

Chapter 12

Testing

Testing is a critical phase ensuring the software operates according to the documented requirements, is free of critical defects, and handles edge cases gracefully.

Unit Testing

Unit testing focused on isolating individual methods, particularly within the Service and Controller layers, to ensure they perform their discrete tasks correctly. For example, the queue number generation logic was tested to ensure that if a queue is empty, the first number assigned is 1, and subsequent additions increment correctly.

Integration Testing

Integration tests verified that different modules of the application communicate seamlessly. A primary focus was the interaction between the Controllers and the Spring Data JPA Repositories. Tests were conducted to ensure that when an Appointment entity is saved, it correctly persists to the MySQL database and that the associated Foreign Keys (Patient ID, Doctor ID) are accurately mapped.

System Testing

System testing involved executing end-to-end workflows from the perspective of the end-user. This included simulating a patient logging in, booking an emergency appointment, observing the appointment appear on the doctor's queue board, and having the doctor mark the appointment as complete.

Sample Test Cases

Test Case 1: Role-Based Access Control - Action: A user logged in with the ROLE_PATIENT attempts to navigate to the URL /admin/dashboard. - **Expected Result:** The system denies access and returns an HTTP 403 Forbidden error or redirects to an access denied page. - **Actual Result:** Access was denied, and the user remained in the patient boundary. (Pass)

Test Case 2: Priority Queue Sorting - Action: Three appointments are booked for the same day. Patient A (Normal), Patient B (Normal), Patient C (Emergency). - **Expected Result:** The doctor's queue should display the order as: Patient C, Patient A, Patient B. - **Actual Result:** The database query sorted by priorityScore DESC successfully placed Patient C at the top of the interface. (Pass)

Test Case 3: Rescheduling Logic - Action: A patient reschedules an appointment from today to a date in the following week. - **Expected Result:** The appointment date updates in the database, and the existing Queue entry for today is marked as CANCELLED and removed from the active view. - **Actual Result:** The backend logic successfully intercepted the date change, updated the appointment, and cancelled the volatile queue entry. (Pass)

Test Case 4: Client-Side Table Search - Action: An admin types "Sanjay" into the search bar on the Manage Doctors page. - **Expected Result:** The JavaScript instantly filters the table to only show rows containing the string "Sanjay". - **Actual Result:** Table filtering worked accurately without requiring a page reload. (Pass)

Chapter 13

Results

The successful compilation, deployment, and testing of the Hospital Virtual Appointment Booking Queue System demonstrate that all specified objectives have been met. The system operates as a cohesive, reliable platform capable of managing complex hospital scheduling dynamics.

Screens

(Note: Visual placeholders for implementation screens)

[Placeholder: Screenshot of the Patient Dashboard showing the live queue position and upcoming appointments]

[Placeholder: Screenshot of the Doctor's 'Today's Queue' view, highlighting the Start, Complete, and Reschedule action buttons]

[Placeholder: Screenshot of the Admin Dashboard displaying global statistics and the system-wide appointment management table]

[Placeholder: Screenshot of the Reschedule Form demonstrating the date and time selection interface]

Discussion

Performance The application demonstrated excellent performance characteristics. The use of Spring Data JPA enabled highly optimized database queries, ensuring that dashboard loads and queue updates occurred in milliseconds. The frontend, heavily utilizing Tailwind CSS, resulted in minimal payload sizes, allowing the interface to render swiftly even on slower network connections.

Security The integration of Spring Security proved highly effective. The rigorous enforcement of endpoint protection ensures that user data is isolated. The transition to BCrypt password hashing guarantees that even in the unlikely event of a database breach, user passwords remain secure. The application securely manages session states, preventing unauthorized hijacking.

User Experience The user experience was significantly enhanced through the application of modern design principles. The implementation of Tailwind CSS provided a clean, professional, and consistent aesthetic. The inclusion of responsive design ensures the application is highly usable on mobile devices, which is critical for patients monitoring their queue status remotely. Furthermore, the use of Toastify for non-blocking notifications provides clear feedback (e.g., "Successfully deleted record") without disrupting the user's workflow.

Chapter 14

Conclusion

The development of the Hospital Virtual Appointment Booking Queue System represents a significant technological upgrade over traditional medical scheduling practices. By migrating the queuing process from crowded hospital waiting rooms to a secure, accessible web platform, the system addresses the core inefficiencies of outpatient management.

Through the implementation of a robust Spring Boot backend, the software successfully automates the complex logistics of appointment booking, queue number generation, and dynamic triage sorting. The introduction of the priority scoring algorithm elegantly solves the historical problem of integrating emergency walk-ins into a pre-scheduled roster without causing administrative chaos.

Furthermore, the dedicated interfaces for Patients, Doctors, and Administrators ensure that each stakeholder is empowered with the exact tools and data they require to perform their roles efficiently. The project successfully demonstrates the application of modern software engineering principles—including MVC architecture, ORM, and responsive web design—to solve real-world logistical problems in the healthcare sector. Ultimately, the system lays the groundwork for a more organized, transparent, and patient-friendly healthcare experience.

Chapter 15

Future Enhancements

While the current system fulfills all core objectives, software development is an iterative process. Several enhancements could be implemented in future versions to further expand the system's utility and integration within a broader hospital environment:

1. **SMS and Email Integration:** Implementing a notification service (using APIs like Twilio or SendGrid) to automatically send SMS or email alerts to patients when their queue number is approaching, or if an appointment is rescheduled by an administrator.
 2. **Electronic Health Records (EHR) Module:** Expanding the system to allow doctors to input detailed consultation notes, upload prescriptions, and view historical medical charts directly within the platform.
 3. **Integrated Payment Gateway:** Incorporating digital payment solutions (e.g., eSewa, Khalti, or Stripe) to allow patients to pay consultation fees online at the time of booking, reducing the workload on hospital cashiers.
 4. **Machine Learning for Wait Times:** Replacing the static estimated waiting time calculation with a machine learning model that analyzes historical consultation durations per specific doctor to provide highly accurate, dynamic wait time predictions.
 5. **Multi-Department Support:** Enhancing the database schema to support complex hospital hierarchies, allowing the categorization of doctors and queues into specific departments (e.g., Cardiology, Neurology) with department-specific administrators.
-

Chapter 16

References

1. Craig Walls, *Spring in Action, Sixth Edition*, Manning Publications, 2022.
2. Laurentiu Spilca, *Spring Security in Action*, Manning Publications, 2020.
3. Vlad Mihalcea, *High-Performance Java Persistence*, Leanpub, 2016.
4. Adam Wathan & Steve Schoger, *Refactoring UI*, 2018.
5. Tailwind CSS Documentation, *Tailwind Labs*, Available at: <https://tailwindcss.com/docs>
6. Thymeleaf Official Documentation, Available at: <https://www.thymeleaf.org/doc/tutorials/3.0/usingthymeleaf.html>

7. Spring Boot Reference Documentation, *Pivotal Software*, Available at:
<https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
8. MySQL 8.0 Reference Manual, *Oracle Corporation*, Available at:
<https://dev.mysql.com/doc/refman/8.0/en/>
9. "Impact of virtual queuing systems on patient waiting times in outpatient clinics",
Journal of Healthcare Engineering, 2021.
10. "The Role of Web-Based Appointment Systems in Modern Healthcare
Administration", *International Journal of Medical Informatics*, 2020.